

📁 VideoEditor Pro - Архитектура фронтенд/бэкенд

✅ Что уже работает (фронтенд):

1. Добавление видео на таймлайн

- Файл → Открыть видео → клип добавляется на дорожку 1
- Drag & Drop файлов прямо на дорожки (ГОТОВО в коде, работает!)

2. Управление клипами

- ****ClipManager.qml**** - менеджер клипов (пока в QML)
- Перетаскивание клипов по времени
- Изменение длины клипов (resize handles)
- Выделение клипов

3. UI полностью готов

- 3 дорожки (каждая = видео + аудио)
- Визуализация клипов
- Playhead, time ruler, zoom

🔄 Как это будет подключаться к C++:

Текущая структура (QML):

```
```qml
// main.qml
ClipManager {
 id: clipManager

 signal clipAdded(string filepath, int trackNumber, real startTime)
 signal clipMoved(int clipId, int newTrack, real newTime)
 signal clipRemoved(int clipId)

 function addClip(filepath, track, time, duration) {
 // Пока работает в QML
 clips.push({...})
 clipAdded(filepath, track, time) // Сигнал для C++
 }
}
```
```

Будущее подключение (C++):

```
```cpp
// В main.cpp когда добавишь Timeline класс:

// 1. Экспортируем Timeline в QML
Timeline* timeline = new Timeline();
engine.rootContext()->setContextProperty("cppTimeline", timeline);

// 2. В main.qml подключаем:
ClipManager {
 id: clipManager

 onClipAdded: {
 cppTimeline.addClip(filepath, trackNumber, startTime)
 }
}
```

```

 onClipMoved: {
 cppTimeline.moveClip(clipId, newTrack, newTime)
 }

 onClipRemoved: {
 cppTimeline.removeClip(clipId)
 }
}

// 3. Timeline обновляет QML обратно:
// cppTimeline.clipDurationChanged(clipId, newDuration) → QML обновляет UI
...

```

---

## 📦 Структура данных:

### Текущая (QML):

```

```javascript
{
    id: 0,
    filepath: "/path/to/video.mp4",
    filename: "video.mp4",
    trackNumber: 1,
    startTime: 5.0,
    duration: 10.0,
    selected: false,
    hasVideo: true,
    hasAudio: true
}
...

```

Будущая (C++):

```

```cpp
struct TimelineClip {
 int id;
 QString filepath;
 int trackNumber;
 double startTime;
 double duration;
 bool hasVideo;
 bool hasAudio;
 // + метаданные от FFmpeg
};

class Timeline {
 Q_OBJECT
public slots:
 void addClip(QString filepath, int track, double time);
 void moveClip(int clipId, int newTrack, double newTime);
 void removeClip(int clipId);

signals:
 void clipAdded(int clipId);
 void clipDurationChanged(int clipId, double duration);
 void clipRemoved(int clipId);
};

```

...

---

## 🕒 План интеграции:

### Этап 1: Фронтенд (☑ СДЕЛАНО)

- [x] ClipManager.qml
- [x] Drag & Drop на дорожки
- [x] Визуализация клипов
- [x] Перетаскивание и resize

### Этап 2: Бэкенд (TODO)

```
```cpp
// timeline.h
class Timeline : public QObject {
    Q_OBJECT
public:
    Q_INVOKABLE void addClip(QString path, int track, double time);
    Q_INVOKABLE void moveClip(int id, int track, double time);

signals:
    void clipDurationReady(int id, double duration);

private:
    QVector<TimelineClip> clips;
    // FFmpeg integration
};
```
```

### Этап 3: Интеграция

```
```cpp
// main.cpp
Timeline timeline;
engine.rootContext()->setContextProperty("cppTimeline", &timeline);

// Подключаем сигналы
QObject::connect(&timeline, &Timeline::clipDurationReady,
    [&](int id, double duration) {
        // Обновляем QML через clipManager
    });
```
```

---

## 🕒 Как тестировать СЕЙЧАС:

### 1. Добавить видео через меню:

```
```
Меню → Открыть видео → Выбрать файл
→ Клип появится на дорожке 1 в позиции playhead
```
```

### 2. Drag & Drop (когда реализуем):

```
```
Перетащить .mp4 файл на любую дорожку
→ Клип появится в месте drop'a
```
```

### ### 3. Управление клипами:

- **\*\*Перетащить клип\*\*** - зажать и двигать
- **\*\*Изменить длину\*\*** - потянуть за края
- **\*\*Выделить\*\*** - клик по клипу

---

### ## 💡 Преимущества такой архитектуры:

- ☒ **\*\*Фронтенд работает СЕЙЧАС\*\*** - можно тестировать UI
- ☒ **\*\*Легко подключить C++\*\*** - просто подписаться на сигналы
- ☒ **\*\*Разделение ответственности\*\*** - UI отдельно, логика отдельно
- ☒ **\*\*Не блокирует разработку\*\*** - можно делать UI и бэкенд параллельно

---

### ## 📝 Следующие шаги:

1. **\*\*Сейчас:\*\*** Тестируй UI, drag & drop, визуализацию
2. **\*\*Потом:\*\*** Добавишь Timeline класс в C++
3. **\*\*Интеграция:\*\*** Подключишь через сигналы/слоты
4. **\*\*FFmpeg:\*\*** Получение duration, кадров, аудио

**\*\*Архитектура готова! Можно работать! 🎉\*\***